

“Live in the Editor”: Low-Tech In-Situ Feedback for Plain-Text Live Programming

Luyu Cheng

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Hong Kong, China
luyu.cheng@connect.ust.hk

Bairui Su

Tisch School of the Arts
New York University
New York City, New York, USA
bs4396@nyu.edu

Lionel Parreaux

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Hong Kong, China
parreaux@ust.hk

Abstract

Programming often benefits from immediate, co-located behavioral feedback such as values, traces, and visual hints. Computational notebooks are a popular way to provide such feedback, but their reliance on cells, execution order, global state, and dedicated interfaces can introduce conceptual and practical overhead in some contexts. In this extended abstract, we explore a low-tech, complementary design point: providing in-situ runtime feedback directly within plain-text source files. Rather than competing with notebooks, the approach trades rich graphical interfaces for low implementation overhead, strong portability, and seamless integration with existing text-based workflows, while retaining much of the immediacy and expressiveness of live feedback. The core of the system is a runtime that incrementally re-evaluates semantically defined code cells and embeds their outputs adjacent to source code as specialized comments. We present a working prototype and outline its design rationale, implementation, and initial evaluation plan.

CCS Concepts

• Human-centered computing → Interactive systems and tools.

Keywords

in-situ feedback, plain-text programming, programming environments, exploratory programming, lightweight tools

ACM Reference Format:

Luyu Cheng, Bairui Su, and Lionel Parreaux. 2026. “Live in the Editor”: Low-Tech In-Situ Feedback for Plain-Text Live Programming. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798318>

1 Introduction

Programming is often easier to learn, explore, and reason about when behavioral feedback—values, traces, and small visualizations—appears immediately and remains co-located with the code that

produces it. Seeing what a program does, right where it is written, supports rapid experimentation and reduces the cognitive gap between intention and behavior.

Computational notebooks, exemplified by Jupyter Notebook [13], Observable [11], and related systems, are a widely used way to provide such feedback. They excel at rich outputs, interactive widgets, and exploratory workflows, and are often the right tool for those goals. At the same time, notebooks introduce additional concepts—cells, execution order, and implicit global state—that can confuse some beginners [3]. They also rely on a dedicated environment and user interface, and may require nontrivial setup, library overhead, or continuous Internet access. These characteristics are not flaws, but tradeoffs that make notebooks better suited for some contexts than others.

In this extended abstract, we explore a *low-tech* alternative: providing immediate, in-situ runtime feedback directly inside plain-text documents. Rather than competing with notebooks, we investigate a complementary design point that trades rich graphical interfaces for low implementation overhead, strong portability, and seamless integration with existing text-based workflows, while retaining much of the responsiveness and expressiveness of live feedback. The current prototype targets JavaScript as its source language. Our design is guided by the following choices.

Plain-text as the medium. The entire notebook is a plain-text buffer. Cells are defined semantically by the system, rather than visually by a specialized user interface. As a result, no dedicated notebook UI is required. The plain-text format *enables* compatibility with any editor that can modify text files, from modern IDEs to terminals accessed over SSH, although the current prototype ships with its own lightweight editor. Syntax highlighting and other visual aids (e.g., color) are provided by the editor, as with any source-code file, and are not embedded in the plain text itself.

Despite its simplicity, plain text remains expressive. Values, documentation, and execution traces can be rendered directly, and Unicode characters (e.g., box-drawing characters or emojis) enable lightweight textual visualizations, without requiring a separate graphical API.

In-situ feedback. Runtime output is written directly adjacent to the code that produces it, using a special form of comments. Unlike notebook output cells, which appear in a separate region below each code cell, outputs here are interleaved within the source text itself. Embedding results in plain text improves portability and diff-ability, making files easier to



This work is licensed under a Creative Commons Attribution 4.0 International License. [CHI EA '26, Barcelona, Spain](https://creativecommons.org/licenses/by/4.0/)

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2281-3/26/04
<https://doi.org/10.1145/3772363.3798318>

inspect, version, and review in collaborative settings using version control systems (e.g., Git).

In our prototype, outputs are produced via a print-like function called `echo`¹, which can display values, record intermediate steps, or emit ASCII-based visualizations.

Immediacy at low cost. Whenever the document is saved, code is re-evaluated by an underlying runtime (comparable to a *kernel* in Jupyter Notebook). The runtime splits the document into cells and incrementally re-executes only changed cells and their dependents.

To support this behavior, the runtime tracks variable definitions and uses them to maintain a dependency graph. Runtime outputs are attributed to their source cells and written back into the text buffer in place, yielding a form of liveness directly inside the editor.

In summary, we explore a lighter-weight corner of the design space for immediate feedback in programming tools. In the remainder of this extended abstract, we describe the system design and implementation (Sections 2–3), situate the work relative to prior approaches and tradeoffs (Section 4), and outline an initial evaluation plan and next steps (Section 5). We present a working prototype alongside this manuscript, *Recho*, as part of the supplementary material. An online demo is available at <https://recho.dev>, with a collection of expressive examples. We invite readers to try it out.

2 Design

We illustrate the design of our system through a concrete authoring workflow: constructing a moon phase calendar for a given year. The example demonstrates how users interact with the system during exploratory programming, and how immediate, in-situ feedback supports inspection, iteration, and refinement. We interleave code snippets with brief explanations of user actions in the editor. For brevity, we show only newly introduced or modified code at each step.

2.1 Case Study: Authoring a Moon Phase Calendar

Step 1: Preparing the Data with Live Inspection. To generate a moon phase calendar for an entire year, we first enumerate all days in the year and compute the moon phase for each day. We begin by defining the first day of the year and the first day of the following year.

```
1 const year = 2025;
2 const beginDate = new Date(year, 0, 1);
3 const endDate = d3.timeYear.offset(beginDate, 1);
```

Users write code as usual. When they are unsure about an intermediate result, they can wrap an expression with the `echo` function and save the document. The runtime evaluates the code and inserts the result as a special comment (prefixed with `//->`) above the expression. For example, to inspect the value of `endDate`, the user could write:

```
4 const endDate = echo(d3.timeYear.offset(beginDate, 1));
```

¹We borrow the name from the Shell command.

After the user presses the save button, the runtime executes the statement and inserts the output before the line:

```
4 //-> Fri Jan 01 2027 00:00:00 GMT+0000
5 const endDate = echo(d3.timeYear.offset(beginDate, 1));
```

The utility function used to compute the date offset is imported from D3 [1]. Because the document is split into semantically defined cells and executed by a runtime, statements do not need to appear in textual execution order. Variables may be defined after they are used, allowing users to freely reorder code—for example, placing final outputs near the top of the document and external imports at the end.

```
6 const d3 = recho.require("d3");
```

Here, `recho.require` is the prototype's module import mechanism, which fetches and caches an external package by name.

Next, we enumerate all days between `beginDate` and `endDate`. To confirm that the boundaries are correct, we inspect the length of the resulting array.

```
7 const days = d3.timeDays(beginDate, endDate);
8 //-> 365
9 echo(days.length);
```

Step 2: Computing the Moon Phase. To keep the example concise, we import an external library, `sunalc`, which computes moon illumination for a given time. When using a new or under-documented library, inspecting return values directly in the editor helps users quickly build understanding.

```
10 const sunalc = recho.require("sunalc");
11 //-> { fraction: 0.21, phase: 0.15, angle: -2.00 }
12 echo(sunalc.getMoonIllumination(new Date()));
```

From the output, we identify the `phase` property and inspect its range across all days in the year.

```
13 //-> [ 0.00, 0.99 ]
14 echo(d3.extent(days.map(
15   d => sunalc.getMoonIllumination(d).phase
16 )));
```

Our goal is to visualize moon phases using Unicode emoji symbols (new moon, waxing crescent moon, first quarter moon, waxing gibbous moon, full moon, waning gibbous moon, last quarter moon, and waning crescent moon), whose code points range from `U+1F311` to `U+1F318`. We therefore map the quantitative range `[0, 1]` to this ordinal range.

```
17 function getMoonEmoji(date) {
18   const index = Math.round(
19     sunalc.getMoonIllumination(date).phase * 8
20   );
21   return String.fromCharCode(
22     0x1f311 + (index === 8 ? 0 : index)
23   );
24 }
```

Step 3: Displaying the Calendar. Finally, we construct a multi-line string in which each line represents a month. This step consists primarily of assembling arrays and formatting strings. Because results are displayed immediately above the corresponding code, users can rapidly adjust the implementation when the output does not match their expectations.

We omit the final formatting adjustments for brevity. Figure 1 shows the completed calendar with month and weekday headers. The full example is available in the supplementary material and the online demo.

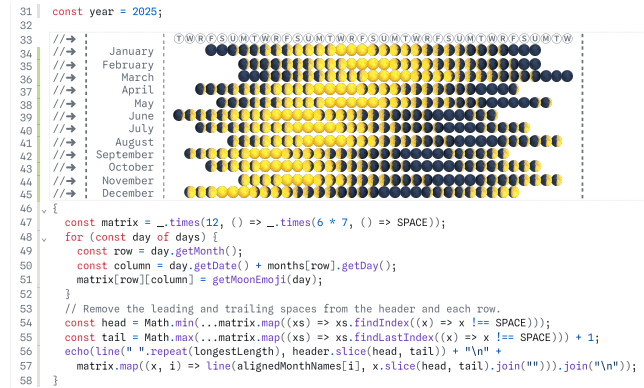


Figure 1: The moon phase calendar rendered as plain text with Unicode emojis in Recho. The definition of helpers is omitted for brevity. The full example is available in the online demo.

2.2 Highlights

This workflow illustrates a low-ceremony interaction model that supports sustained exploratory flow. Users write code in a standard plain-text editor and remain there throughout the process. When they are unsure about an intermediate result—such as the structure or range of a return value—they can insert `echo` to inspect it immediately, with the output appearing directly adjacent to the relevant code. This avoids context switches to external documentation or debugging tools and keeps attention localized. After inspection, users may remove `echo` or retain it as lightweight, executable documentation embedded in the source file. The interaction is continuous and keyboard-centered, allowing users to effectively “live” in the editor while iteratively exploring program behavior.

2.3 Design Choices

We highlight several design decisions that shape the system.

What Defines a Cell? In computational notebooks, a *cell* is the minimal unit of execution. In our design, each top-level statement constitutes a cell. This choice yields a finer-grained dependency graph and enables more precise incremental re-execution.

A Single Output Primitive. The `echo` function serves as a general-purpose output mechanism. It displays values directly, and switches to a verbatim display mode for multi-line strings, omitting quotation marks and escape sequences. This

makes it suitable for rendering ASCII-based and Unicode-based visualizations.

Deterministic Execution via a Dependency Graph. The runtime maintains a dependency graph in which nodes correspond to cells and edges capture variable dependencies. This structure enables deterministic execution order and allows only affected cells to be re-evaluated when the document changes, supporting immediacy without requiring a global re-run.

3 Implementation

We implemented a working prototype, *Recho*, including a basic editor and document storage. The core of the system is a lightweight, editor-agnostic runtime (analogous to the *kernel* in Jupyter Notebook). The runtime treats the document as both input and output: it reads a plain-text source file, evaluates code incrementally, and writes results back into the same file as structured output comments. This design allows the system to integrate with existing text-based workflows without requiring a specialized editor or file format.

3.1 Runtime Pipeline

Whenever the document changes and feedback is expected, the runtime executes the following pipeline.

Slice the document into cells. The document is parsed into an abstract syntax tree, and each top-level statement is extracted as a cell. Cells are matched against the existing set to detect additions, removals, or modifications.

Compute dependencies. For each cell, the runtime performs lightweight static analysis to identify declared variables and free variables. If a cell *A* references a variable defined in cell *B*, an edge is added from *B* to *A* in the dependency graph.

Incrementally re-evaluate affected cells. When a cell changes, only that cell and its dependents are re-executed, following the partial order induced by the dependency graph.

Rewrite outputs into the document. Outputs produced during execution are attributed to their source cells and written back into the document as structured comments adjacent to the corresponding code.

3.2 Output Attribution

Determining where the output of an `echo` call should appear is non-trivial. A purely syntactic association—attributing each `echo` to the cell in which it is written—leads to unintuitive behavior when helper functions are involved. For example, if a function defined in one cell emits output but is invoked from another, users expect the output to appear near the call site. To support this behavior, the runtime attributes output dynamically. During execution, it tracks the *currently executing cell* and assigns all `echo` calls to that cell, regardless of where they are defined syntactically. The attribution state is updated when execution enters a new cell and restored afterward, ensuring that outputs appear where they are conceptually produced.

3.3 Asynchronous Operations

The computational graph architecture also simplifies handling asynchronous operations. In JavaScript, asynchronous functions return

Promise objects, typically requiring explicit propagation of asynchronous control flow. In Recho, when a cell evaluates to a **Promise**, dependent cells are scheduled to execute only after the promise resolves. The outputs are written back into the document incrementally as the asynchronous operations complete.

4 Related Work and Discussion

Live programming environments. Live programming environments interleave editing and evaluation to reduce the temporal and perceptual gap between code and behavior, often by displaying concrete runtime values alongside source code as it changes [12, 19]. Systems such as Hazel formalize semantics for incomplete programs to preserve live feedback even when code is syntactically or type incomplete [12]. Other work integrates probes, traces, or continuously refreshed execution feedback directly into the editor to support debugging and program comprehension through co-located runtime information [10, 16]. Example-based approaches embed live examples into general-purpose code to expose runtime state for selected expressions in larger programs [14]. These systems typically rely on custom editors or language runtimes, whereas our approach seeks similar always-on, in-situ feedback within unmodified plain-text files. More lightweight designs have explored writing execution results back into plain-text source files—for example, a diff-based approach to interactive compiler debugging and testing for MLscript [4].

Computational notebooks. Computational notebooks such as Jupyter tightly couple code and output through cell-based execution, supporting exploratory and literate programming workflows [5]. While persistent global state and flexible execution order enable rapid iteration, they can also lead to hidden dependencies, ordering errors, and reproducibility issues as notebooks scale [2, 9]. Prior work addresses these issues through dependency tracking and lineage analysis (e.g., nbsafety) [9], scoped multi-cell regions to reduce global state [15], and mechanisms for persisting interactive analysis steps in reusable execution histories [6]. In contrast to these notebook-centric designs, our system provides immediate feedback without requiring a notebook interface or a global, mutable notebook state.

Editor-integrated feedback. A complementary line of work augments traditional text editors with in-situ visualizations of runtime values or analysis results. Inline displays of variable state and execution behavior have been shown to improve task performance and perceived understanding compared to conventional debugging tools [7]. Educational live programming systems similarly visualize runtime values inline to aid tracing and debugging, while managing the risk of information overload [8]. Beyond dynamic behavior, interactive static analysis frameworks deliver incremental dataflow and control-flow feedback directly in the editor with low latency [17]. Other systems for creative coding blur the boundary between source and interface by allowing code fragments to both display and manipulate state [18]. Our work aligns with these editor-integrated approaches, but emphasizes a low-tech representation in which runtime feedback is encoded as structured comments, prioritizing portability across languages and tooling.

5 Future Work

We have informally shared the system and examples with people from various backgrounds, including students, researchers, and practitioners. Feedback was generally positive, particularly regarding the low-ceremony workflow and the ability to view runtime results directly alongside code in a plain-text editor. Several participants noted the appeal of keeping exploration “inside the editor” for lightweight debugging, learning, and text-based visualizations. While anecdotal, these observations suggest the approach is understandable and worth deeper study.

We plan a small formative evaluation to examine when in-situ, text-first feedback is effective and where it falls short. Participants with basic programming experience will complete short exploratory tasks (e.g., debugging, tracing, or constructing simple textual visualizations) using our prototype and a baseline workflow such as a standard editor or notebook. Beyond evaluation, we plan to refine error presentation to better communicate result staleness, expand the set of built-in examples, and explore applying the same architecture to languages beyond JavaScript.

We expect the approach to be most effective for self-contained exploratory tasks such as data transformation, algorithm prototyping, and small visualizations, where immediate value inspection provides the most benefit. Scaling to multi-file codebases is a natural next step; this would likely require cross-file dependency tracking and a project-level runtime, which we leave to future work.

Acknowledgments

We thank April Wang, Liwenhan Xie, Fei Chen, and Ashley Zhang for their valuable feedback on earlier drafts of this work. Jack B. Du created several examples with Recho that are exhibited in the system, and Mike Bostock provided inspiration in the early stages of this project.

References

- [1] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. 2011. D³ Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics* 17, 12 (2011), 2301–2309. doi:10.1109/TVCG.2011.185
- [2] C. Brown, Hamed Alhoori, and D. Koop. 2023. Facilitating Dependency Exploration in Computational Notebooks. *Proceedings of the Workshop on Human-In-the-Loop Data Analytics* (2023). doi:10.1145/3597465.3605222
- [3] Souti Chattopadhyay, Ishita Prasad, Austin Z. Henley, Anita Sarma, and Titus Barik. 2020. What’s Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI ’20). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3313831.3376729
- [4] Luyu Cheng and Lionel Parreaux. 2024. Diff-based interactive compiler debugging and testing. In *Proceedings of the 10th Workshop on Live Programming (LIVE 2024)*. Accepted workshop essay at LIVE 2024, available at <https://live-2024.pages.dev>.
- [5] H. Fangohr, T. Kluyver, and M. DiPierro. 2021. Jupyter in Computational Science. *Comput. Sci. Eng.* 23 (2021), 5–6. doi:10.1109/mcse.2021.3059494
- [6] Kiran Gadhave, Zach Cutler, and Alexander Lex. 2024. Persist: Persistent and Reusable Interactions in Computational Notebooks. *Computer Graphics Forum* 43 (2024). doi:10.1111/cgf.15092
- [7] J. Hoffswell, Arvind Satyanarayan, and Jeffrey Heer. 2018. Augmenting Code with In Situ Visualizations to Aid Program Understanding. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (2018). doi:10.1145/3173574.3174106
- [8] Ruanqianqian Huang, Kasra Ferdowsi, Ananthi Selvaraj, Adalbert Gerald Soosai Raj, and Sorin Lerner. 2022. Investigating the Impact of Using a Live Programming Environment in a CS1 Course. *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1* (2022). doi:10.1145/3478431.3499305
- [9] Stephen Macke, Hongpu Gong, D. Lee, Andrew Head, Doris Xin, and Aditya G. Parameswaran. 2020. Fine-Grained Lineage for Safer Notebook Interactions. *Proc. VLDB Endow.* 14 (2020), 1093–1101. doi:10.14778/3447689.3447712

- [10] Sean McDirmid. 2013. Usable live programming. (2013), 53–62. doi:10.1145/2509578.2509585
- [11] Observable, Inc. 2018. Observable Notebooks. <https://observablehq.com>. Accessed: 2026-01-19.
- [12] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2018. Live functional programming with typed holes. *Proceedings of the ACM on Programming Languages* 3 (2018), 1 – 32. doi:10.1145/3290327
- [13] Project Jupyter. 2015. Jupyter Notebook. <https://jupyter.org>. Accessed: 2026-01-19.
- [14] David Rauch, Patrick Rein, Stefan Ramson, Jens Lincke, and R. Hirschfeld. 2019. Babylonian-style Programming: Design and Implementation of an Integration of Live Examples into General-purpose Source Code. *ArXiv abs/1902.00549* (2019). doi:10.22152/programming-journal.org/2019/3/9
- [15] Eric Rawn and Sarah E. Chasins. 2025. Pagebreaks: Multi-Cell Scopes in Computational Notebooks. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (2025). doi:10.1145/3706598.3713620
- [16] Patrick Rein, Christian Flach, Stefan Ramson, Eva Krebs, and Robert Hirschfeld. 2024. Broadening the View of Live Programmers: Integrating a Cross-Cutting Perspective on Run-Time Behavior into a Live Programming Environment. *Art Sci. Eng. Program.* 8 (2024). doi:10.22152/programming-journal.org/2024/8/13
- [17] Idriss Riouak, Niklas Fors, G. Hedin, and Christoph Reichenbach. 2024. IntraJ: an on-demand framework for intraprocedural Java code analysis. *International Journal on Software Tools for Technology Transfer* 26 (2024), 687 – 705. doi:10.1007/s10009-024-00771-0
- [18] Charles Roberts, M. Wright, and J. Kuchera-Morin. 2015. Beyond editing: extended interaction with textual code fragments. (2015), 126–131. doi:10.5281/zenodo.1179164
- [19] S. Tanimoto. 2013. A perspective on the evolution of live programming. *2013 1st International Workshop on Live Programming (LIVE)* (2013), 31–34. doi:10.1109/live.2013.6617346